AGILE SOFTWARE DEVELOPMENT DOCUMENT

Epic: Customer Service

Project: E-Commerce Customer Service Management System
Development Methodology: Agile – Scrum
Epic Name: Customer Service
Priority: High

1. Introduction

The Customer Service module is designed to provide customers with a convenient platform to manage their orders, returns, refunds, addresses, and Prime membership.

The system will allow customers to track their packages, edit or cancel orders, return or exchange products, print return labels, manage delivery addresses, and manage their Prime membership.

The project follows the Agile Scrum methodology, where development is divided into smaller, manageable tasks and delivered incrementally through sprints. This approach allows continuous feedback, testing, and improvements throughout the development process.

2. Epic Description

Epic: Customer Service

As an e-commerce customer, I want to manage my orders, returns, refunds, addresses, and Prime membership from a single platform so

that I can resolve my shopping-related requirements easily and efficiently.

3. Features

The Customer Service Epic consists of the following features:

Feature ID	Feature Name	Description
F01	Your Orders	Allows customers to view and manage their orders.
F02	Returns and Refunds	Allows customers to return or exchange products and manage refunds.
F03	Manage Address	Allows customers to add, update, delete, and manage delivery addresses.
F04	Manage Prime	Allows customers to view and manage their Prime membership.

4. User Stories

User stories describe the functionality from the customer's perspective.

4.1 Your Orders Feature

User Story 1: Track Packages

As a customer, I want to track my packages so that I can know the current delivery status and estimated arrival of my orders.

Acceptance Criteria:

- The customer should be able to view their placed orders.
- The customer should be able to select an order and view its tracking details.
- The system should display the current order status.
- The system should display estimated delivery information when available.
- The tracking information should be updated based on the latest available shipment details.

User Story 2: Edit or Cancel Orders

As a customer, I want to edit or cancel my orders so that I can make changes or cancel an order when required.

Acceptance Criteria:

- The customer should be able to view their existing orders.
- The customer should be able to edit eligible order details.
- The customer should be able to cancel eligible orders.
- The system should validate whether an order is eligible for modification or cancellation.
- The customer should receive confirmation after a successful update or cancellation.
- Orders that have already been shipped or delivered should follow the applicable cancellation and return policies.

4.2 Returns and Refunds Feature

User Story 3: Return and Exchange Items

As a customer, I want to return or exchange purchased items so that I can resolve issues with products that I no longer need or that do not meet my expectations.

Acceptance Criteria:

- The customer should be able to view eligible purchased items.
- The customer should be able to select a product for return or exchange.
- The customer should provide a reason for the return.
- The system should validate the return eligibility.
- The customer should be able to view the return request status.
- The system should provide refund or exchange information.

User Story 4: Print Labels

As a customer, I want to generate and print return shipping labels so that I can send products back conveniently.

Acceptance Criteria:

- The customer should be able to generate a return label for an approved return request.
- The system should generate a downloadable return label.
- The customer should be able to print the generated label.
- The label should contain the necessary shipping information.
- The system should prevent label generation for ineligible return requests.

5. Product Backlog

The Product Backlog contains all the development activities required to implement the Customer Service Epic.

Backlog ID	User Story	Priority
US01	Track Packages	High
US02	Edit or Cancel Orders	High
US03	Return and Exchange Items	Medium
US04	Print Labels	Medium
US05	Manage Address	Medium
US06	Manage Prime	Medium

6. Sprint Planning

The project will be divided into multiple sprints. Each sprint will focus on delivering a specific set of functionalities.

Sprint 1: Track Packages

Sprint Goal: Develop the package tracking functionality that allows customers to view their order and shipment status.

Task ID	Task Description	Technology	Estimated Hours
T01	Build the React UI	React.js	5
T02	Build the FastAPI Layer	Python, FastAPI	6
T03	Build the Database Schema	SQL / Database	8
T04	Write Unit Test Cases	Pytest, React Testing Library	4

T05	Configure CI/CD Pipeline for the Feature	GitHub Actions	2
	Total		**25 Hours**

Sprint 1 Deliverable: A working package tracking module integrated with the backend and database, with unit tests and CI/CD configuration.

Sprint 2: Edit Orders

Sprint Goal: Implement the functionality that allows customers to edit or cancel eligible orders.

Task ID	Task Description	Technology	Estimated Hours
T06	Build the React UI	React.js	5
T07	Build the FastAPI Layer	Python, FastAPI	6
T08	Build the Database Schema	SQL / Database	8
T09	Write Unit Test Cases	Pytest, React Testing Library	4
T10	Configure CI/CD Pipeline for the Feature	GitHub Actions	2
	Total		**25 Hours**

Sprint 2 Deliverable: A working order editing and cancellation module with backend integration, database support, unit tests, and CI/CD configuration.

7. Task Breakdown and Responsibilities

Each user story is divided into smaller development tasks to make implementation and progress tracking easier.

7.1 Track Packages

1. **React UI Development – 5 Hours**
    - Design the order tracking page.
    - Display order details and shipment status.
    - Create components for tracking information.
    - Integrate the UI with backend APIs.
2. **FastAPI Development – 6 Hours**
    - Create API endpoints to retrieve order details.
    - Implement package tracking APIs.
    - Handle API requests and responses.
    - Add input validation and error handling.
3. **Database Schema Development – 8 Hours**
    - Design the orders table.
    - Create the shipment tracking table.
    - Establish relationships between customers, orders, and shipments.
    - Store delivery status and tracking information.
4. **Unit Testing – 4 Hours**
    - Test order retrieval APIs.
    - Test tracking status functionality.
    - Validate successful and unsuccessful API responses.
    - Test frontend components.
5. **CI/CD Configuration – 2 Hours**
    - Configure automated build workflows.
    - Run unit tests during the pipeline.
    - Validate successful builds before deployment.

7.2 Edit Orders

1. **React UI Development – 5 Hours**
    - Design the order management interface.
    - Add edit and cancel options.

    - Create forms to update eligible order details.
    - Display order modification confirmation messages.
2. **FastAPI Development – 6 Hours**
    - Create APIs to update order details.
    - Create APIs to cancel orders.
    - Implement order eligibility validation.
    - Handle errors and API responses.
3. **Database Schema Development – 8 Hours**
    - Design or update the order management tables.
    - Add fields required for order modification.
    - Maintain order cancellation status.
    - Implement database relationships and constraints.
4. **Unit Testing – 4 Hours**
    - Test order update APIs.
    - Test order cancellation functionality.
    - Validate order eligibility conditions.
    - Test invalid requests and error handling.
5. **CI/CD Configuration – 2 Hours**
    - Configure automated build and test workflows.
    - Run unit tests during code integration.
    - Validate the pipeline before deployment.

8. Agile Scrum Roles

The following roles are involved in the Agile development process:

Role	Responsibility
Product Owner	Defines requirements, manages the Product Backlog, and prioritizes user stories.
Scrum Master	Facilitates Scrum meetings and removes obstacles faced by the development team.
Development Team	Develops the React UI, FastAPI backend, and database components.

Testing Team	Performs unit testing and verifies the implemented functionality.

9. Agile Ceremonies

The Scrum framework includes the following activities:

1. Sprint Planning

- Select user stories from the Product Backlog.
- Define sprint goals.
- Break user stories into smaller tasks.
- Estimate the effort required for each task.

2. Daily Stand-up Meeting

- Discuss completed work.
- Explain the tasks planned for the day.
- Identify blockers or challenges.

3. Sprint Review

- Demonstrate the completed functionality.
- Collect feedback from stakeholders.
- Identify improvements or additional requirements.

4. Sprint Retrospective

- Discuss what went well during the sprint.
- Identify challenges faced by the team.
- Plan improvements for the next sprint.

10. Definition of Done (DoD)

A user story is considered complete when:

- All the planned development tasks are completed.

- React UI is successfully integrated with FastAPI.
- Database operations work correctly.
- All required unit tests pass.
- Code is reviewed and merged into the main development branch.
- CI/CD pipeline executes successfully.
- Acceptance criteria are satisfied.
- No critical or high-priority defects remain unresolved.

11. Technology Stack

Component	Technology
Frontend	React.js
Backend	Python, FastAPI
Database	PostgreSQL
Testing	Pytest, React Testing Library
Version Control	Git, GitHub
CI/CD	GitHub Actions
Development Methodology	Agile Scrum

12. Project Estimation

The following table summarizes the estimated development effort for the selected user stories.

User Story	Estimated Hours
Track Packages	25
Edit or Cancel Orders	25

Total Estimated Effort	**50 Hours**

Note: The above estimation covers only the Track Packages and Edit or Cancel Orders user stories. The remaining features and user stories will be estimated and scheduled in subsequent sprints.

13. Expected Outcomes

After completing the planned sprints:

- Customers will be able to track their packages.
- Customers will be able to view shipment and delivery information.
- Customers will be able to edit eligible orders.
- Customers will be able to cancel eligible orders.
- The application will provide a user-friendly interface.
- Backend APIs will securely process order-related operations.
- Automated testing and CI/CD will improve software quality and delivery.

14. Conclusion

The Customer Service Epic will be developed using the Agile Scrum methodology to ensure incremental development, continuous testing, and regular feedback.

The initial two sprints will focus on the Your Orders feature, specifically Track Packages and Edit or Cancel Orders. Each user story is divided into manageable tasks covering frontend development, backend development, database design, unit testing, and CI/CD configuration.

The remaining features, including Returns and Refunds, Manage Address, and Manage Prime, will be taken up in subsequent development iterations.

By following Agile practices, the project aims to deliver a reliable, maintainable, and customer-friendly e-commerce service management system.